

WHERE SPARSITY MATTERS: SHAPING ACTIVATIONS FOR EFFICIENT TRANSFORMER EXECUTION

Anonymous authors

Paper under double-blind review

ABSTRACT

Training-time activation pressure (P) induces smaller activation magnitudes, while thresholding (T) maps small values to exact zeros. Where should these interventions be placed to reduce inference latency at an acceptable validation-loss cost? We study broad thresholding recipes (T_4, T_7) with local or global pressure under matched Pythia-14M pretraining conditions, coupling activation-sparsity measurements with specialized GPU execution. We find that higher model-wide sparsity does not necessarily yield lower latency, and that broader interventions can incur substantially larger loss penalties. Sparsity and execution diagnostics instead converge on two sites. Pressure applied only at the FFN hidden activation (h) also increases sparsity at the unpressured attention-output activation (z), a nonlocal response we call sparsity spillover. Conditional kernel ablations identify h and z as the profitable skipping paths. This motivates a narrower intervention, T_2/P_h , which pressures h and thresholds only h, z . At moderate threshold ($\kappa = 0.1$), it reaches validation loss 5.151 versus 5.209 for the base model. With weights and thresholding fixed, exploiting its h, z zeros reduces latency by 12.9% relative to the same implementation with both skipping paths disabled. At 70M, T_2/P_h retains the lowest validation loss among the tested threshold topologies, but the 14M optimized kernel does not transfer directly. After further kernel optimization, T_7/P_h at $\kappa = 0.5$ shows a 17.4% latency reduction from specialized h, z execution relative to native dense replacements. These results show that activation sparsification is a training–execution co-design problem: sparsity must be induced where its structure can be profitably exploited at inference time.

1 INTRODUCTION

Activation sparsity can improve transformer inference efficiency when specialized kernels can skip computation and memory accesses of zero activations. Recently, Cetin et al. (2026) has successfully demonstrated inference speedups from train-time pressure and specialized kernel design (Cetin et al., 2026). This result motivates a follow-up question: how broadly and aggressively should train-time interventions be applied throughout the transformer to induce exploitable sparsity while reducing latency at an acceptable quality cost?

Existing approaches differ in where and how they introduce sparsity. TEAL applies post-hoc, training-free thresholding to pretrained models; ProSparse and Cetin et al. (2026) apply ℓ_1 regularization to feed-forward (FFN) activations during training; and Q-Sparse applies top- k sparsification to both FFN and attention activations (Liu et al., 2024; Song et al., 2024; Wang et al., 2024). However, these studies do not isolate the effects of different train-time sparsity interventions under a common training protocol. Under matched pretraining conditions using models from the Pythia family (Biderman et al., 2023), we systematically vary the coverage and intensity of train-time sparsity interventions and measure their maximum sparsity ceiling ($S_{\text{model}}^{\text{max}}$), realized model-wide sparsity (S_{model}), and execution efficiency.

We define an intervention by the placement of pressure (P) and thresholding (T). To keep the intervention space tractable, for P we introduce Adam-orthogonalized ℓ_1 activation pressure (OL1), limiting conflict with the task update while avoiding an additional pressure-strength sweep. For T we use a fixed-threshold nonlinearity with $\kappa \in 0, 0.01, 0.05, 0.1, 0.5$. The P and T placements are independent, so a recipe such as T_7/P_{all} specifies their respective site sets, while sweeping κ

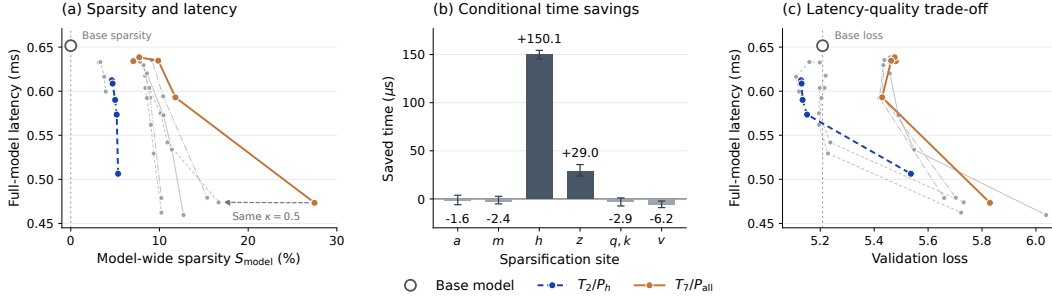


Figure 1: **Sparsity, execution savings and quality on Pythia-14M.** (a) Model-wide logical sparsity S_{model} versus full-model latency. The arrow connects T_7/P_{all} to T_7/P_h at $\kappa = 0.5$: 27.48% and 16.66% sparsity yield similar latency. (b) Conditional execution ablations on T_7/P_{all} at $\kappa = 0.5$ identify h, z as the profitable skipping sites. Saved time is latency with one skipping path disabled minus latency with all paths enabled, holding the checkpoint and thresholding fixed. Whiskers span differences between process means, not confidence intervals; these savings are not additive. (c) Validation loss versus latency exposes the quality cost of broader sparsification. Panels (a,c) show the same 40 checkpoints: T_2/P_h is blue, T_7/P_{all} orange, and other recipes gray. Hollow markers and vertical guides denote Base; lines connect settings within recipes. Timings use BF16 on an RTX 5090, batch one and 2,048 tokens; panel (b) uses a separate controlled session.

controls sparsification aggressiveness and traces the recipe’s trade-off curve. Each model is then connected with an specialized kernel that connects its sparsity directly with execution benefit.

Under extensive 14M ablations, we first observe that broader intervention coverage can substantially extend attainable model-wide sparsity without translating into lower latency. At $\kappa = 0.5$, expanding pressure and thresholding from four to seven sites more than doubles model-wide sparsity, from 12.71% to 27.48%, while latency slightly increases from 0.459 to 0.473 ms (Figure 1(a)). Conditional kernel ablations then identify h and z as the main latency-relevant skipping paths, with the largest benefit at h (Figure 1(b)). Sparsity measurements reveal a related nonlocal effect: applying pressure only at h also increases sparsity at the thresholded z site, despite z receiving no direct pressure (Figure 3). We call this effect *sparsity spillover*. OL1 geometry further distinguishes h : applying pressure beyond h increases both the pressure-update norm and the component removed by orthogonalization, with the strongest effect at q, k, v , indicating that these sites are less susceptible to pressure than h (Figure 6).

Together, these ablations point to h and z as the most useful sparsification sites: they are more responsive to intervention and their sparsity translates into latency reduction, even when model-wide sparsity is lower than with broader recipes. This motivates a targeted T_2/P_h recipe: apply pressure only at h , exploit its spillover to z , and threshold only h and z . At 14M and $\kappa \leq 0.1$, T_2/P_h achieves both lower validation loss than the dense control and lower full-model latency (0.573 vs. 0.655 ms; Figure 1(c)).

2 RELATED WORK

2.1 SPARSIFICATION INTERVENTIONS

Post-hoc thresholding removes small activations at inference-time of a pre-trained model. While CATS thresholds small-magnitude activations within FFNs (Lee et al., 2024), TEAL extends it to attention (Liu et al., 2024). These studies motivate training-time interventions to adapt activation distributions to sparse computation and improve the trade-off between quality and sparsity.

ReLU Strikes Back studies the effect of activations functions in language models (Mirzadeh et al., 2023), showing clear sparsity gains on the use of ReLUs. ProSparse combines progressive ℓ_1 pressure and thresholding at FFNs during continued training (Song et al., 2024). It does not, however, extend its interventions to attention and restrict the analysis to local FFN sparsity. Cetin et al. (2026) extends ProSparse setup to pre-training and propose specialized kernels. These studies show that

training can reshape the quality–sparsity trade-off, but they’re restricted to a single local intervention at FFN sites.

Q-Sparse extend sparsification beyond FFNs. It applies top- k selection to both attention and FFN projection inputs (Wang et al., 2024), but with no ℓ_1 pressure. Its top- k operation introduces, however, additional overhead due to the sorting operation. Spark Transformer tries to address it with statistical top- k (You et al., 2025), but we take a simpler approach and use a constant thresholding non-linearity. We argue that when the intervention is introduced during training, the network can adapt itself. Additionally, we evaluate the complementary effect of regularization pressure on both attention and FFNs.

2.2 INFERENCE SPEEDUP

Turning activation sparsity into speedup requires a specialized kernel adapted to the model architecture. TEAL accelerates batch-one autoregressive decoding, reporting up to $1.8\times$ speedup at 50% sparsity (Liu et al., 2024). Cetin et al. (2026) address batched matrix multiplication with their TwELL kernel, and report $1.3\times$ speedup for batch inference. Spark likewise couples its sparsity mechanism with an implementation that reports decoding gains (You et al., 2025). In each setting, skipped computation and memory access must compesate the additional overhead introduced by the sparse operations. This development has been increasingly accelerated by the use of coding agents, through iterative kernel development with execution and profiling feedback (Ouyang et al., 2025; Luo et al., 2026). We take a related approach and use GPT-6 Astra to develop a specialized kernel based on our training interventions (OpenAI, 2026).

3 METHODOLOGY

3.1 SPARSIFICATION INTERVENTIONS

A sparsification intervention specifies where activations are thresholded (thresholding sites $\mathcal{N}$) and where activation pressure is applied (pressure sites $\mathcal{P}$). The sets $\mathcal{N}$ and $\mathcal{P}$ need not coincide, and the thresholding and pressure functions may take different forms. Together, these choices define a combinatorial space of training interventions. We restrict this space to keep the computational cost manageable while addressing the central research questions.

For thresholding, we avoid the overhead of exact top- k selection (Wang et al., 2024; You et al., 2025) and instead use one-sided and symmetric fixed-threshold operators $G_{+,\kappa}(x) = x\mathbf{1}\{x \geq \kappa\}$ and $G_{\pm,\kappa}(x) = x\mathbf{1}\{|x| \geq \kappa\}$, with $\kappa \geq 0$. The constant κ controls the strength of the intervention and is held fixed within each training condition. At $\kappa = 0$, $G_{+,\kappa}$ reduces to ReLU, whereas $G_{\pm,\kappa}$ reduces to the identity. We use one-sided thresholding at projection inputs, where signed weight products can recover either sign, and symmetric thresholding for attention operands to better preserve their signed dot-product distribution.

For activation pressure, ℓ_1 averages across targeted sites and layers the mean absolute activation at each site after any thresholding. Ordinary L1 pressure minimizes $\mathcal{L}_{\text{task}} + \lambda\ell_1$, where λ controls the pressure intensity (Song et al., 2024; Cetin et al., 2026). To limit conflict between the ℓ_1 pressure and the task update while reducing the need to tune an additional hyperparameter, we introduce orthogonal ℓ_1 pressure (OL1), inspired by gradient-conflict projection methods (Yu et al., 2020; Hsieh et al., 2024). OL1 adds a separately preconditioned pressure correction after the task-only AdamW update, projects away its component opposing the adaptive task direction, and caps its norm relative to that direction. At a fixed optimizer state, once the norm budget is active, further increasing λ leaves the pressure correction unchanged up to numerical stabilizers. To keep the multisite intervention study tractable, we use a common $\lambda = 1$ and budget $b = 1$ and directly measure whether the budget becomes active during training. Appendix A.1 gives the full formulation, a brief comparison and shows that budget saturation depends strongly on the target set.

Figure 2 locates the intervention sites within the Pythia architecture. The variables a and m denote the normalized inputs to the attention and FFN branches, respectively; h is the FFN hidden activation; z is the concatenated attention context before the output projection; and q , k , and v are the query, key, and value operands. Attention probabilities are denoted by p . We apply $G_{+,\kappa}$ at selected sites in $\{a, m, h, z\}$ and $G_{\pm,\kappa}$ at selected sites in $\{q, k, v\}$. When h is targeted, the thresholding

nonlinearity replaces GELU. Table 1 summarizes the studied thresholding and pressure assignments and their sparsity ceilings across model sizes. Section 3.2 defines these ceilings.

Table 1: Intervention recipes and analytic sparsity ceilings. T_1 replaces GeLU with ReLU at h . Pressure uses OL1. For $T_2/T_4/T_7$, we sweep $\kappa \in 0, .01, .05, .1, .5$ with $\lambda = 1$ and $b = 1$; the matched 70M latency sweep for T_2/P_h stops at $\kappa = 0.1$, with its additional $\kappa = 0.5$ endpoint reported separately in Appendix D.1; the 14M T_1/P_1 ablation instead sweeps $\lambda \in .05, .1, .5, 1$ at $b = 1$. Ceilings are $100 \cdot \mathcal{S}_{\text{model}}^{\max}$ at 2,048 tokens. Executed recipes by model size are listed in Appendix C.3.

Recipe	Threshold sites $\mathcal{N}$	Pressure sites $\mathcal{P}$	Ceiling (%)		
			14M	70M	410M
T_0/P_0 (Base Model)	$\emptyset$	$\emptyset$	0.00	0.00	0.00
T_1/P_0	h	$\emptyset$	4.28	12.35	24.93
T_1/P_1	h	h	4.28	12.35	24.93
T_2/P_h	h, z	h	5.35	15.44	31.16
T_4/P_0	a, m, h, z	$\emptyset$	12.83	37.06	74.78
T_4/P_h	a, m, h, z	h	12.83	37.06	74.78
T_4/P_{all}	a, m, h, z	$\mathcal{N}$	12.83	37.06	74.78
T_7/P_0	a, m, h, z, q, k, v	$\emptyset$	29.95	49.42	87.25
T_7/P_h	a, m, h, z, q, k, v	h	29.95	49.42	87.25
T_7/P_{all}	a, m, h, z, q, k, v	$\mathcal{N}$	29.95	49.42	87.25

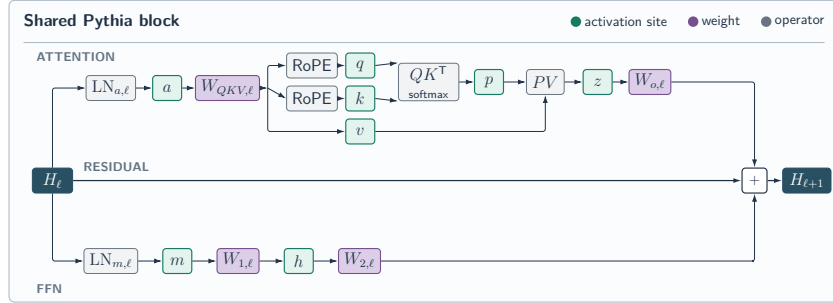


Figure 2: Intervention sites in a shared Pythia block. The attention and FFN branches use separate LayerNorms and join the residual path. Green nodes denote activation sites, and purple nodes denote learned weights. Table 1 specifies the thresholding and pressure assignments.

3.2 SPARSITY MEASURE AND ARCHITECTURAL REACH

Activation sparsity is the fraction of zero activations at a site. We define *model-wide sparsity* as $\mathcal{S}_{\text{model}} = N_0/N_{\text{model}}$, where N_0 counts scalar multiplications with a zero activation operand in Pythia-block matrix products, and N_{model} counts all scalar multiplications in those blocks and the final output projection. We aggregate counts over validation data, excluding masked attention pairs.

To quantify intervention potential a priori, we define the *model-wide sparsity ceiling* as $\mathcal{S}_{\text{model}}^{\max} = N_{\text{reach}}(\mathcal{N})/N_{\text{model}}$, where $N_{\text{reach}}(\mathcal{N})$ counts scalar multiplications guaranteed a zero activation operand if all selected thresholding sites are 100% sparse. Thus, $\mathcal{S}_{\text{model}}^{\max}$ is the maximum sparsity directly attributable to $\mathcal{N}$. It depends on architecture, selected sites, and sequence length. Appendix B gives the precise accounting and definitions.

4 EXPERIMENTAL STUDY

We pretrain Pythia-14M, 70M, and 410M from random initialization (Biderman et al., 2023) on MiniPile (Kaddour, 2023). Each run receives 712 optimizer steps and 1.493 billion input tokens. Within each size, runs share initialization, data order and training budget. We evaluate loss and $\mathcal{S}_{\text{model}}$ on the full validation set. Appendix C.3 gives the full protocol.

4.1 EXTENSIVE ABLATIONS AT 14M

We compare intervention breadth and strength at 14M, separating aggressive thresholding ($\kappa = 0.5$) from moderate settings ($\kappa = 0.05, 0.1$). **Broader coverage increases attainable sparsity without consistently reducing latency.** At $\kappa = 0.5$, T_7/P_{all} more than doubles the sparsity of T_4/P_{all} (27.48% versus 12.71%), yet takes 0.473 versus 0.459 ms (Table 2). Within T_7 , broadening pressure from P_h to P_{all} adds 10.82 percentage points of sparsity with nearly unchanged latency, while loss rises from 5.732 to 5.829. All seven aggressive recipes reduce latency relative to Base but increase loss.

Table 2: **Sparsity and latency under aggressive thresholding.** All seven thresholded 14M recipes at $\kappa = 0.5$. Ceiling and sparsity are $100S_{\text{model}}^{\text{max}}$ and $100S_{\text{model}}$; loss is full-validation cross-entropy. All latencies, including Base, use the specialized 14M kernel on RTX 5090 (BF16, batch one, 2,048 tokens, full logits), with three timing processes per checkpoint. Measurements retain their original sessions; small differences do not establish a reliable ordering.

Recipe	Ceiling (%)	Sparsity (%)	Latency (ms)	Loss
Base (kernel)	0.00	0.00	0.652	5.209
T_2/P_h	5.35	5.34	0.506	5.536
T_4/P_0	12.83	10.22	0.479	5.660
T_4/P_h	12.83	10.23	0.462	5.723
T_4/P_{all}	12.83	12.71	0.459	6.038
T_7/P_0	29.95	15.39	0.479	5.703
T_7/P_h	29.95	16.66	0.474	5.732
T_7/P_{all}	29.95	27.48	0.473	5.829

Moderate thresholding with local pressure offers a better quality-latency balance. At both moderate thresholds, P_h gives lower loss and latency than P_{all} under both T_4 and T_7 (Table 3). At $\kappa = 0.05$, T_4/P_h reaches 0.562 ms with loss 5.196, versus Base’s 0.652 ms and 5.209. Narrowing thresholding to T_2/P_h improves loss further to 5.134, but takes 0.590 ms at 5.00% sparsity. Thus, the preferred breadth depends on the quality cost one accepts; maximizing model-wide sparsity alone does not select the best execution trade-off. We next examine where sparsity forms and which sites save time.

4.2 SPILLOVER: LOCAL PRESSURE INDUCES NONLOCAL SPARSITY

Pressure at h increases sparsity at z under both T_4 and T_7 . At $\kappa = 0.05$, P_h raises h sparsity from 74–85% to 94–99% across layers (Figure 3). Sparsity at the unpressured z site also rises in every layer: from 71–85% to 83–92% under T_4 , and from 70–87% to 84–94% under T_7 . This spillover is therefore not confined to four-site thresholding, and suggests that z is responsive to the changes induced by pressure at h . This is evidence of the network representation globally re-adapting itself from a local intervention.

Broader pressure reveals unequal sparse susceptibility across sites. Under T_4/P_{all} , extending pressure beyond h increases sparsity across the targeted sites, with z reaching 99.74–99.85% sparsity in layers 2–4. Such high sparsification may contribute to the higher validation loss (5.490 versus 5.196 for P_h). Extending pressure further under T_7/P_{all} produces a different pattern: sparsity at q, k, v increases but remains only 3–13%, while h and z become less sparse than under P_h in every layer. The OL1 geometry in Figure 6 is consistent with this asymmetry. Pressure outside h produces larger pre-cap correction norms and larger components opposing the task update, and the effect is

Table 3: **Quality–latency trade-offs at moderate thresholds.** All seven thresholded 14M recipes at $\kappa \in \{0.05, 0.1\}$, with the same metrics and timing protocol as Table 2.

Recipe	κ	Ceiling (%)	Sparsity (%)	Latency (ms)	Loss
Base (kernel)	—	0.00	0.00	0.652	5.209
T_2/P_h	0.05	5.35	5.00	0.590	5.134
	0.1	5.35	5.17	0.573	5.151
T_4/P_0	0.05	12.83	8.21	0.630	5.434
	0.1	12.83	8.95	0.593	5.420
T_4/P_h	0.05	12.83	9.04	0.562	5.196
	0.1	12.83	9.34	0.529	5.229
T_4/P_{all}	0.05	12.83	10.46	0.573	5.490
	0.1	12.83	11.39	0.534	5.548
T_7/P_0	0.05	29.95	9.13	0.635	5.438
	0.1	29.95	10.43	0.594	5.429
T_7/P_h	0.05	29.95	10.13	0.575	5.195
	0.1	29.95	10.90	0.542	5.237
T_7/P_{all}	0.05	29.95	9.86	0.635	5.463
	0.1	29.95	11.80	0.593	5.429

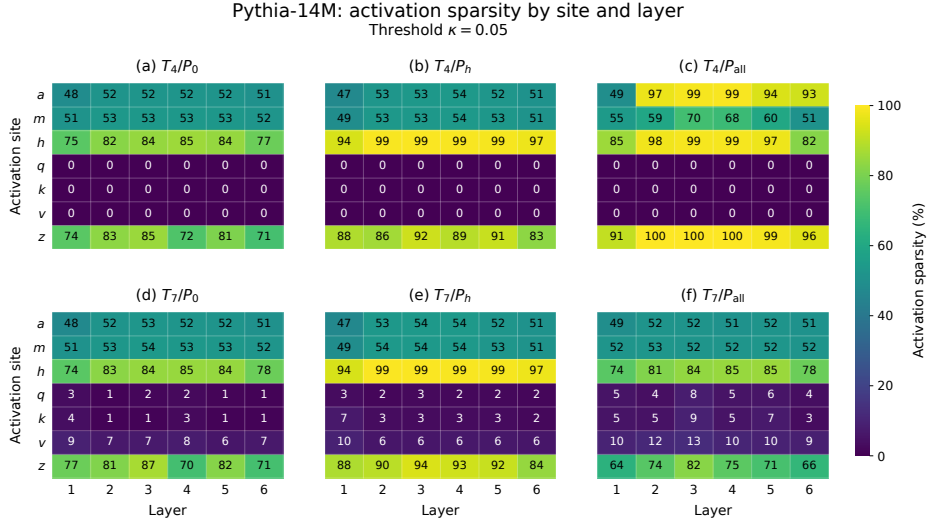


Figure 3: **Activation sparsity by site and layer in Pythia-14M.** The six threshold/pressure recipes at $\kappa = 0.05$: T_4 (top) and T_7 (bottom), each under P_0 , P_h and P_{all} . Each cell gives the percentage of zero activations over all complete validation sequences at one site and layer. Values are rounded to whole percentages, so 0 or 100 does not establish a completely dense or all-zero tensor.

strongest for T_7/P_{all} . At $\kappa = 0.05$, the median removed component rises from 0.15–0.17% under P_h to 1.12% for T_4/P_{all} and 66.84% for T_7/P_{all} , while the corresponding pre-cap norm ratios rise from 0.08–0.10 to 0.28 and 46.00. Thus, broadening pressure increasingly triggers OL1 projection and norm capping, potentially reducing the effective pressure reaching h and z . Together with the spillover effect, these results indicate that h and z are more susceptible to sparsification than the additional sites.

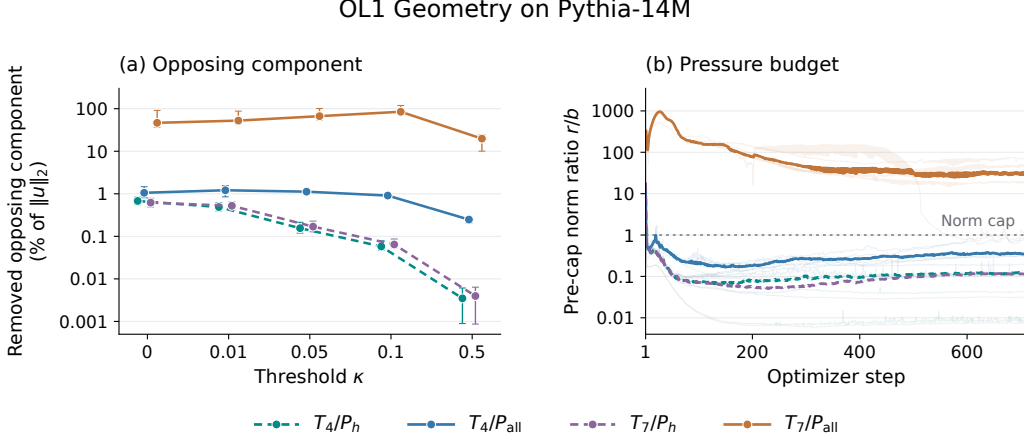


Figure 4: **OL1 geometry depends on pressure placement at 14M.** (a) Median removed opposing component, $100\rho_{\text{opp}}$; bars span the 25th–75th percentiles over 712 training steps per condition, not confidence intervals. (b) Pre-cap norm ratio r/b : faint curves show individual thresholds, and bold curves their pointwise median. The dotted line marks the norm cap.

4.3 WHEN SPARSITY TRANSLATES TO SPEEDUP

We next test whether the sites with the most susceptibility to sparsity also respond most strongly to execution time. Table 4 isolates each skipping path at 14M by disabling it while keeping all the other paths fixed. It reports site-sparsity, *bypass* (the percentage of omitted matrix instructions) and conditional saved time for T_7/P_{all} at $\kappa = 0.5$. We test over this recipe because it’s the sparser variant. The result is aligned with the susceptibility observation: **the measurable savings comes only from h and z** : 150.1 μs and 29.0 μs , respectively.

Table 4: **Conditional latency savings:** 14M T_7/P_{all} , $\kappa = 0.5$. Latency is measured per 2,048-token sequence on RTX 5090 (BF16, batch one). Saved time is full-model latency with one skipping path off minus latency with all paths on; positive values indicate a benefit (reduced latency). Brackets give [minimum, maximum] savings across all off/on pairs of three independent timing runs. Each run reports the geometric mean over 64 sequences.

Operation	Activation sparsity (%)	Bypass (%)	Saved time (μs)	Process span (μs)
a : QKV projection	75.6	8.4	−1.6	[−6.0, +3.8]
m : FFN-up	68.6	0.0	−2.4	[−5.1, +2.7]
h : FFN-down	99.9	94.5	+150.1	[+145.4, +154.4]
z : attention-output projection	99.9	99.6	+29.0	[+23.7, +35.7]
q, k : QK scores	93.5 / 94.5	58.0	−2.9	[−7.3, +1.1]
v : PV product	98.7	67.2	−6.2	[−9.0, −2.2]

Activation zeros pooled over layers and validation sequences; q/k reported separately.

Sparsity reduces latency when avoided work exceeds the overhead of detecting and skipping it. At h, z , the sparse kernel checks 8×16 activation tiles (i.e. sub-matrices) before loading their associated weights, so empty tiles avoid both weight reads and matrix multiplication. Applying the same strategy at a and m doesn’t have an expressive bypass, and instead increased full-model

latency by 17.2% and 27.4%, because the skipped work does not surpass the additional overhead. We speculate that the relation between local sparsity and bypass is not linear, and only over a critical level it translates in a structure that produces high-zero tiles with high probability.

Attention operations q, k, v follows FlashAttention’s tiled computation (Dao et al., 2022). In our kernel implementation, the zero checks happens only after loading the operands. Skipping a matrix instruction therefore avoids the arithmetic without avoiding those reads and softmax operation. Additionally, softmax turns zero scores into nonzero probabilities, so q, k sparsity does not automatically propagate to p . Enabling qk/pv skipping together raises full-model latency from 0.514 to 0.521 ms despite substantial bypass. So, for this implementation, attention’s arithmetic savings do not offset its overhead, while h/z benefit from avoiding weight reads as well as computation.

4.4 RESULTS AT 70M

Increasing model size lowers loss but raises latency. Figure 5 compares the 14M and 70M recipes: native Base loss falls from 5.209 to 4.100, while latency rises from 0.655 to 1.611 ms. As at 14M, aggressive interventions trade quality for lower latency. For example, T_7/P_{all} at $\kappa = 0.5$ increases loss above Base by 1.116 at 70M, versus 0.621 at 14M. Yet T_2/P_h retains the lowest loss among the three threshold topologies at every matched κ . At 70M and $\kappa \leq 0.1$, it reaches 14.63–15.21% model-wide sparsity against a 15.44% ceiling, with loss 4.129–4.193. This concentration of sparsity at h, z , with a smaller quality penalty, is consistent with their susceptibility extending across the two scales.

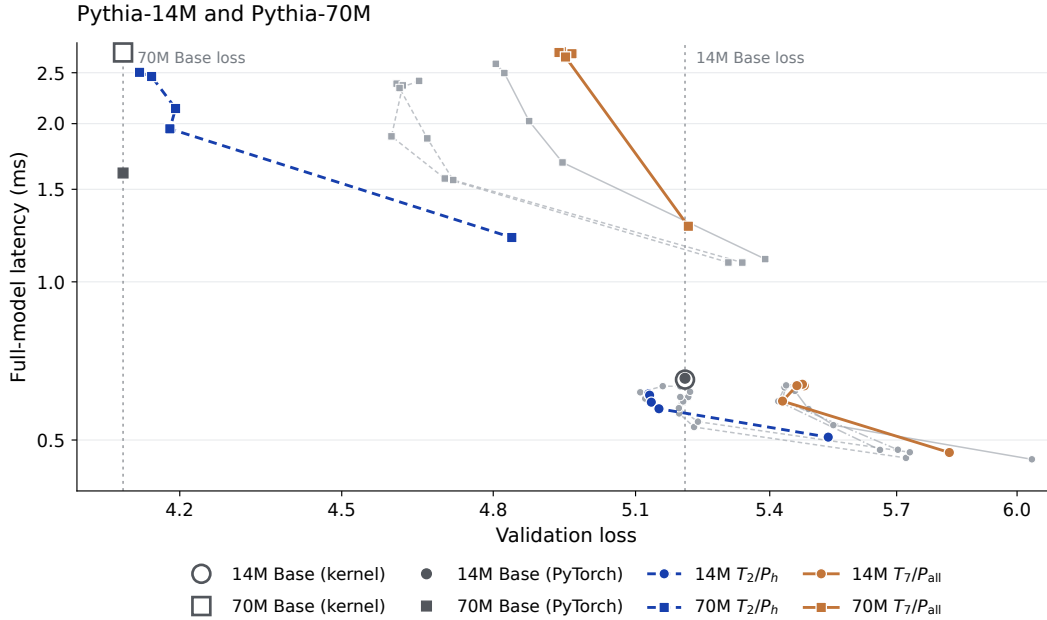


Figure 5: **Validation loss and full-model latency across model sizes.** Circles denote 14M and squares 70M; T_2/P_h is blue, T_7/P_{all} orange, and other recipes gray. Lines connect settings within each recipe and size. Both axes are logarithmic. Each Base checkpoint appears under kernel (hollow) and native PyTorch (filled) execution; vertical guides mark Base loss. The 64 intervention points use the specialized kernels for their respective sizes. Timings use RTX 5090, BF16, batch one, 2,048 tokens and full logits, retaining the original measurement sessions.

Threshold strength controls the quality–latency trade-off at both scales. For T_2/P_h , increasing κ from 0 to 0.1 reduces latency from 0.612 to 0.573 ms at 14M (0.039 ms), and from 2.504 to 1.956 ms at 70M (0.549 ms). The larger absolute reduction at 70M is a comparison across trained checkpoints, not an isolated skipping effect. Increasing κ further to 0.5 brings 70M latency to 1.214 ms but raises loss from 4.182 to 4.838. This large gap between sampled operating points motivates testing intermediate thresholds; their quality–latency balance remains unmeasured.

The 14M kernel does not transfer efficiently to 70M. On the 70M Base checkpoint, the original port takes 3.325 ms versus native PyTorch’s 1.611 ms, an additional 1.714 ms. A further search over 74 candidate implementations improves sparse h, z execution and dense components, reducing Base latency to 2.732 ms; the remaining gap is still 1.121 ms. The fastest optimized recipe, T_7/P_h at $\kappa = 0.5$, reaches 1.088 ms, 32.5% below native Base, but its loss is 5.337, or 1.237 higher. This is an end-to-end engineering comparison across different checkpoints, not a gain attributable entirely to sparsity. The lower-loss T_2/P_h at $\kappa = 0.1$ remains slower than native execution even on the same checkpoint (1.956 versus 1.662 ms; Appendix E.4).

Sparse execution remains useful, but its implementation requires adaptation. In a separate session on the same 70M T_7/P_h checkpoint at $\kappa = 0.5$, replacing only the specialized h, z path with native operations raises latency from 1.128 to 1.365 ms: a conditional 0.237 ms (17.4%) benefit from the specialized path, including fusion and layout effects. The corresponding skip-disabled control is reported in Appendix E.4. Thus, useful sparse structure can persist while kernel efficiency fails to transfer, even within one model family. Our results motivate kernels adapted to model size and workload, rather than assuming that a published implementation transfers unchanged. Training interventions and kernel execution must therefore be co-designed: training should induce sparse structures that the target kernel can exploit at an acceptable quality cost.

5 DISCUSSION AND CONCLUSION

Our results suggest that sparsification should be understood as a joint training-and-deployment problem rather than as a property of activation distributions alone. First, **local pressure does not imply a local response**: applying pressure only at the FFN hidden site can alter sparsity at other sites, showing that the network can reorganize its internal representations in response to a localized training intervention. More broadly, **sparsity is useful only if the resulting zero structure can be exploited by the inference system**. S_{model} is best viewed as a training-time proxy for computational opportunity, not as a direct predictor of latency. In the controlled 14M measurements, the largest latency gains come from (h, z) sites, which were the ones more susceptible to sparsification, with higher achieved sparsity. At 70M, the 14M implementation required further optimization; the specialized h, z path then reduces latency by 17.4% relative to native h, z operations on the same checkpoint, while dense-kernel improvements also contribute to the end-to-end gains (Appendix E.4). One hypothesis is that we must achieve a high ($> 90\%$) level of site-layer sparsity, so the zeros gets a exploitable tiled structure with higher probability.

Transformers appear sufficiently flexible to adapt their internal representations to architectural and optimization constraints, suggesting that larger gains may come from **designing training objectives around inference requirements**, rather than first maximizing a generic sparsity metric and only afterward attempting to accelerate it. The same principle may extend beyond activation sparsity: training could be used to shape representations for properties required by weight sparsity, quantization, batched execution, or other hardware-specific constraints. In this view, deployment requirements are not merely constraints imposed after training, but potential signals for deciding what properties the network should learn.

Several limitations define the scope of these conclusions and point to concrete directions for further work. Our scaling evidence is limited: although the 410M experiments demonstrate that high model-wide sparsity remains attainable, the fixed token budget leaves 410M in a substantially different optimization regime, so it should be interpreted as a stress test rather than a clean scaling experiment. Our runtime study is also restricted to batch-one, full-sequence inference on an RTX5090; we do not study how sparsity interacts with batched inference, cached decoding, or other hardware.

In addition, the full intervention space is combinatorial, and under the available compute budget we evaluate only a selected set of threshold and pressure placements rather than an exhaustive factorial study. The nonlocal response visible in Figure 3 motivates interventions concentrated at h and z . The T_2/P_h results at 14M support this direction: they preserve much of the execution benefit while avoiding thresholding at sites whose sparsity is expensive in quality or unprofitable for the current kernel. Whether this balance transfers depends on the implementation as well as the sparse structure. Related extensions include learning or tuning different thresholds κ at different sites and directly sparsifying attention probabilities after softmax, which could expose work in pv without relying on q/k zeros to propagate through softmax. More generally, rather than asking where the model can

be made sparse, a productive systems-guided approach is to decompose full-model inference into individual operations, identify where latency and removable work are concentrated, determine what structured representation would allow those operations to be skipped efficiently, and then design training-time interventions that encourage precisely that structure.

AI USE STATEMENT

We used OpenAI Codex with GPT-6 Astra to assist with experimental design refinement, mathematical exposition, implementation and debugging of training and evaluation code, GPU-kernel development, experiment execution and monitoring, analysis and interpretation of results, figure preparation, and manuscript drafting and revision. The authors directed the research and approved experimental designs. AI-assisted implementations were checked through tests and comparisons with reference computations; the kernel validation procedures and their limitations are documented in Appendix E. The authors take responsibility for the final text, code, results and scientific claims.

REFERENCES

- Stella Biderman, Hailey Schoelkopf, Quentin Gregory Anthony, Herbie Bradley, Kyle O’Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, Aviya Skowron, Lintang Sutawika, and Oskar van der Wal. Pythia: A Suite for Analyzing Large Language Models Across Training and Scaling. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 2397–2430. PMLR, 2023. URL <https://proceedings.mlr.press/v202/biderman23a.html>.
- Edoardo Cetin, Stefano Peluchetti, Emilio Castillo, Akira Naruse, Mana Murakami, and Llion Jones. Sparser, Faster, Lighter Transformer Language Models. *arXiv preprint arXiv:2603.23198*, 2026. URL <https://arxiv.org/abs/2603.23198>.
- Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *arXiv preprint arXiv:2205.14135*, 2022. URL <https://arxiv.org/abs/2205.14135>.
- Yu-Guan Hsieh, James Thornton, Eugene Ndiaye, Michal Klein, Marco Cuturi, and Pierre Ablin. Careful with that Scalpel: Improving Gradient Surgery with an EMA. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 19085–19100. PMLR, 2024. URL <https://proceedings.mlr.press/v235/hsieh24a.html>.
- Jean Kaddour. The MiniPile Challenge for Data-Efficient Language Models. *arXiv preprint arXiv:2304.08442*, 2023. URL <https://arxiv.org/abs/2304.08442>.
- Donghyun Lee, Je-Yong Lee, Genghan Zhang, Mo Tiwari, and Azalia Mirhoseini. CATS: Contextually-Aware Thresholding for Sparsity in Large Language Models. In *Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=v3w2a7EInO>.
- James Liu, Pragaash Ponnusamy, Tianle Cai, Han Guo, Yoon Kim, and Ben Athiwaratkun. Training-Free Activation Sparsity in Large Language Models. *arXiv preprint arXiv:2408.14690*, 2024. URL <https://arxiv.org/abs/2408.14690>.
- Ilya Loshchilov and Frank Hutter. Decoupled Weight Decay Regularization. In *International Conference on Learning Representations*, 2019. URL <https://arxiv.org/abs/1711.05101>.
- Mao Luo, Hongbin Li, Feng Lin, Hanling Yi, and Zhe Huang. Agentic Kernel Optimization: Generating State-of-the-Art GPU Kernels Without Hand-Written CUDA. *arXiv preprint arXiv:2608.14560*, 2026. URL <https://arxiv.org/abs/2608.14560>.
- Iman Mirzadeh, Keivan Alizadeh, Sachin Mehta, Carlo C. Del Mundo, Oncel Tuzel, Golnoosh Samei, Mohammad Rastegari, and Mehrdad Farajtabar. ReLU Strikes Back: Exploiting Activation Sparsity in Large Language Models. *arXiv preprint arXiv:2310.04564*, 2023. URL <https://arxiv.org/abs/2310.04564>.

- OpenAI. GPT-6 Astra: A New Generation of Intelligence. OpenAI, September 2026. URL <https://openai.com/index/gpt-6-astra/>. Accessed September 19, 2026.
- Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. KernelBench: Can LLMs Write Efficient GPU Kernels? In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 47356–47415. PMLR, 2025. URL <https://proceedings.mlr.press/v267/ouyang25a.html>.
- Chenyang Song, Xu Han, Zhengyan Zhang, Shengding Hu, Xiyu Shi, Kuai Li, Chen Chen, Zhiyuan Liu, Guangli Li, Tao Yang, and Maosong Sun. ProSparse: Introducing and Enhancing Intrinsic Activation Sparsity within Large Language Models. *arXiv preprint arXiv:2402.13516*, 2024. URL <https://arxiv.org/abs/2402.13516>.
- Hongyu Wang, Shuming Ma, Ruiping Wang, and Furu Wei. Q-Sparse: All Large Language Models can be Fully Sparsely-Activated. *arXiv preprint arXiv:2407.10969*, 2024. URL <https://arxiv.org/abs/2407.10969>.
- Chong You, Kan Wu, Zhipeng Jia, Lin Chen, Srinadh Bhojanapalli, Jiaxian Guo, Utku Evci, Jan Wassenberg, Praneeth Netrapalli, Jeremiah J. Willcock, Suvinay Subramanian, Felix Chern, Alek Andreev, Shreya Pathak, Felix Yu, Prateek Jain, David E. Culler, Henry M. Levy, and Sanjiv Kumar. Spark Transformer: Reactivating Sparsity in FFN and Attention. *arXiv preprint arXiv:2506.06644*, 2025. URL <https://arxiv.org/abs/2506.06644>.
- Tianhe Yu, Saurabh Kumar, Abhishek Gupta, Sergey Levine, Karol Hausman, and Chelsea Finn. Gradient Surgery for Multi-Task Learning. In *Advances in Neural Information Processing Systems*, 2020. URL <https://arxiv.org/abs/2001.06782>.

A INTERVENTION DEFINITIONS

A.1 ACTIVATION PRESSURE AND OL1

Objective and update. For J activation tensors A_j at the sites and layers targeted by $\mathcal{P}$, each with n_j entries after any thresholding, define

$$\ell_1(\theta; \mathcal{P}) = \frac{1}{J} \sum_{j=1}^J \frac{1}{n_j} \sum_{i=1}^{n_j} |A_{j,i}|. \quad (1)$$

Each tensor has equal weight, so changing $\mathcal{P}$ changes both the targets and normalization. Ordinary L1 applies AdamW (Loshchilov & Hutter, 2019) to the globally clipped gradient of $\mathcal{L}_{\text{task}} + \lambda \ell_1$. OL1 separates task and pressure updates: task-only AdamW is followed by a pressure correction that removes the opposing component and limits its norm. Pressure therefore does not directly enter Adam’s moment estimates.

Both gradients are evaluated at the same parameters and averaged over the same microbatches. The clipped task gradient updates AdamW’s bias-corrected moments $\hat{m}_t, \hat{v}_t$; the pressure gradient $g_1 = \nabla_{\theta} \ell_1$ is unclipped and has no momentum. Using the same preconditioner for both directions gives

$$u = \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon_A}, \quad w = \frac{g_1}{\sqrt{\hat{v}_t} + \epsilon_A}. \quad (2)$$

Norms and inner products include parameters with both gradients and initialized Adam state. For $c = \langle u, w \rangle$, $q = \|u\|_2^2$ and stabilizer $\epsilon > 0$,

$$\tilde{w} = \begin{cases} w - \frac{c}{q + \epsilon} u, & c < 0 \text{ and } q > \epsilon, \\ w, & \text{otherwise.} \end{cases} \quad (3)$$

Aligned pressure is retained. For weight $\lambda \geq 0$ and relative norm budget $b > 0$,

$$r = \frac{\lambda \|\tilde{w}\|_2}{\|u\|_2 + \epsilon}, \quad s = \min \left\{ 1, \frac{b}{r + \epsilon} \right\} \quad (r > 0), \quad (4)$$

with $s = 1$ when $r = 0$. After task-only AdamW, each parameter group p receives $\theta_p \leftarrow \theta_p - \alpha_p \lambda s \tilde{w}_p$, where α_p is its learning rate. Before learning-rate scaling and ignoring stabilizers, the correction norm is $\min\{\lambda \|\tilde{w}\|_2, b \|u\|_2\}$. At a fixed optimizer state, increasing λ has no further effect once the cap is active.

OL1 combines conflict projection (Yu et al., 2020) and a smoothed task direction (Hsieh et al., 2024) in Adam’s preconditioned coordinates. The stabilizer makes orthogonality approximate. This calculation excludes group learning rates and decoupled weight decay; avoiding opposition to the Adam direction does not guarantee unchanged task or validation loss.

L1 versus OL1. At 14M T_1/P_1 , matched initialization, data order and training budget yield similar results (Table 5). Sparsity differs by less than 0.02 percentage points at every λ . OL1 has slightly lower loss at $\lambda \in \{0.05, 0.1, 0.5\}$ and higher loss at $\lambda = 1$. One seed cannot establish whether these small loss differences persist across seeds.

Table 5: **L1 and OL1 at T_1/P_1 on Pythia-14M.** Final validation loss and model-wide sparsity after 712 matched training steps; OL1 uses $b = 1$. Both metrics cover all 338 complete validation blocks (500 documents; 1,444 tail tokens excluded). One initialization per condition.

λ	L1		OL1	
	S_{model} (%)	Val. loss	S_{model} (%)	Val. loss
0.05	3.1416	5.2061	3.1452	5.1981
0.1	3.3363	5.1655	3.3386	5.1594
0.5	3.7876	5.1127	3.7680	5.1102
1	3.9493	5.1023	3.9384	5.1210

Update geometry and budget saturation. We measure $\rho_{\text{opp}} = \max(0, -c)/(q + \epsilon)$, the removed component’s norm relative to $\|u\|_2$ before capping. All recorded steps satisfy $q > \epsilon$; aligned steps contribute zero. Figure 6 shows ρ_{opp} and r/b for the 20 14M OL1 conditions in Figure ??, using all 712 updates per condition at $\lambda = b = 1$. The cap is active ($s < 1$) on 96.99% of T_7/P_{all} steps, compared with 0.51% for T_4/P_{all} and 0.25% for either P_h recipe. The diagnostics are consistent with stronger pressure–task conflict when attention sites are included, but do not isolate each site’s contribution.

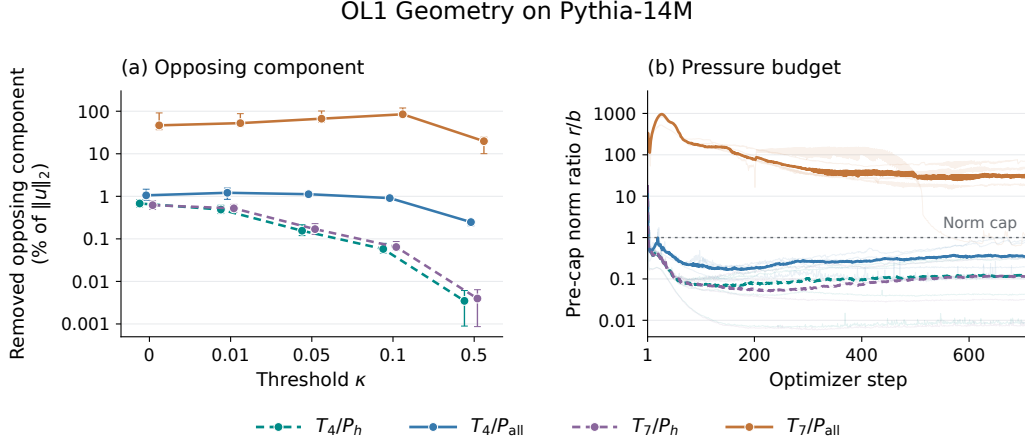


Figure 6: **OL1 geometry depends on pressure placement at 14M.** (a) Median removed opposing component, $100\rho_{\text{opp}}$; bars span the 25th–75th percentiles over 712 training steps per condition, not confidence intervals. (b) Pre-cap norm ratio r/b : faint curves show individual thresholds, and bold curves their pointwise median. The dotted line marks the norm cap. Colors and line styles match Figure ??; both y-axes are logarithmic, and $\lambda = b = 1$ throughout.

B SPARSITY ACCOUNTING

B.1 ACTIVATION AND MODEL-WIDE SPARSITY

Activation sparsity is the fraction of zero entries at a site, pooling counts across layers and validation sequences before dividing.

Model-wide sparsity counts scalar multiplications with a zero activation input:

$$\mathcal{S}_{\text{model}} = \frac{N_0}{N_{\text{model}}}. \quad (5)$$

N_0 counts these multiplications within Pythia blocks; N_{model} counts all multiplications in those blocks and the final vocabulary projection. The latter contributes only to the denominator. Counts are pooled across layers and validation sequences; percentages are $100\mathcal{S}_{\text{model}}$.

For a block’s linear projection from d_{in} to d_{out} features over n rows, the denominator contribution is $nd_{\text{in}}d_{\text{out}}$. Each zero input contributes d_{out} to N_0 , one multiplication per output feature. Zero weights do not count as activation sparsity.

In attention, both operands are activations. We count each multiplication once if either or both are zero. For queries q , keys k , values v and attention probabilities p , one head in one sequence contributes

$$N_{0,\text{QK}} = \sum_{t=1}^T \sum_{u=1}^t \sum_{j=1}^{d_h} \mathbf{1}\{q_{tj} = 0 \vee k_{uj} = 0\}, \quad (6)$$

$$N_{0,\text{PV}} = \sum_{t=1}^T \sum_{u=1}^t \sum_{j=1}^{d_h} \mathbf{1}\{p_{tu} = 0 \vee v_{uj} = 0\}. \quad (7)$$

Here T is sequence length, d_h the head dimension, t the query position and u the key/value position. Restricting $u \leq t$ excludes masked future positions from both counts. We sum over heads, layers and validation sequences, using the actual operands after thresholding and intervening transformations.

Normalization, softmax, activation functions, additions and memory transfers are excluded, so $\mathcal{S}_{\text{model}}$ does not directly measure latency or speedup.

B.2 MODEL-WIDE SPARSITY CEILING

Assume all activations at the thresholding sites $\mathcal{N}$ are zero. If $N_{\text{reach}}(\mathcal{N})$ counts the multiplications that must then have a zero activation input, the ceiling is

$$\mathcal{S}_{\text{model}}^{\text{max}} = \frac{N_{\text{reach}}(\mathcal{N})}{N_{\text{model}}}. \quad (8)$$

The denominator is unchanged, and each multiplication is counted once. The numerator includes downstream activations forced to zero: setting $v = 0$ also makes $z = 0$, affecting both PV and the attention-output projection. Zero attention scores do not imply zero softmax probabilities.

The ceiling depends only on architecture, selected sites and sequence length. Natural zeros outside those sites can raise measured $\mathcal{S}_{\text{model}}$ above it; the ceiling does not predict the associated quality cost. Appendix C.2 gives the counts for each Pythia size.

C EXPERIMENTAL SETUP DETAILS

C.1 PYTHIA ARCHITECTURE AND INTERVENTION SITES

Table 6 gives the shapes and positions of the sites in Figure 2. Here T is sequence length, d hidden width, d_f FFN width and H the number of heads, each with dimension $d_h = d/H$. Sites and threshold are shared across layers.

Table 6: Activation shapes and intervention positions for one sequence.

Site	Shape	Following matrix product	Position
a	$T \times d$	QKV projection	After attention LayerNorm
m	$T \times d$	FFN up projection	After FFN LayerNorm
h	$T \times d_f$	FFN down projection	Activation-function output
q, k	$H \times T \times d_h$	Attention scores	After RoPE
v	$H \times T \times d_h$	Probability-value	After the QKV split
z	$T \times d$	Attention-output projection	Concatenated attention context

C.2 ARCHITECTURE-SPECIFIC SPARSITY CEILINGS

For one sequence, each Pythia block uses $3Td^2$ scalar multiplications in the QKV projection, Td^2 in the attention-output projection, Tdd_f in each FFN projection, and $dT(T+1)/2$ in each of QK and PV . With L layers and vocabulary size V_{vocab} , the totals are

$$\begin{aligned} C_{\text{block}} &= T(4d^2 + 2dd_f) + dT(T+1), \\ C_{\text{model}} &= LC_{\text{block}} + TdV_{\text{vocab}}. \end{aligned} \quad (9)$$

Sequence count cancels from the ceiling ratio. For $d_f = 4d$, per-layer counts in the ceiling numerator are 0 for T_0/P_0 , $4Td^2$ for T_1/P_0 and T_1/P_1 , $12Td^2$ for T_4 , and C_{block} for T_7 . Multiplying by L/C_{model} gives $\mathcal{S}_{\text{model}}^{\text{max}}$, independent of pressure placement.

Table 1 uses $V_{\text{vocab}} = 50,304$ and $(L, d) = (6, 128), (6, 512), (24, 1,024)$ for 14M, 70M and 410M, respectively, with the common sequence length in Table 7.¹

¹Pinned architecture configurations: 14M, 70M, and 410M.

C.3 TRAINING AND EVALUATION

All sizes use the Pythia-14M-deduped tokenizer and seeds of 1234 for initialization and data order, with one seed per condition. AdamW settings are shared: $\beta = (0.9, 0.95)$, $\epsilon = 10^{-8}$, weight decay 0.1 and gradient clipping at norm 1. Attention and hidden dropout are zero. Training uses 712 updates with 1% linear warmup followed by cosine learning-rate decay. Table 7 gives the learning rates, microbatch sizes and shared effective batch. Gradients accumulate over the listed microbatches before each optimizer update.

Table 7: **Training settings across model sizes.** Effective batch size equals microbatch size times gradient accumulation. The common training budget gives fewer tokens per parameter at larger sizes.

Setting	14M	70M	410M
Parameters	14,067,712	70,426,624	405,334,016
Peak learning rate	10^{-3}	10^{-3}	3×10^{-4}
Minimum learning rate	10^{-4}	10^{-4}	3×10^{-5}
Microbatch (sequences)	32	4	4
Gradient accumulation (microbatches)	32	256	256
Effective batch (sequences/update)	1,024	1,024	1,024
Sequence length	2,048	2,048	2,048
Tokens per parameter	106.14	21.20	3.68

All sizes include the T_0/P_0 and T_1/P_0 controls. The 14M study adds T_1/P_1 with L1 or OL1, and T_4 and T_7 with P_0 , P_h or P_{all} . Both 14M and 70M also include T_2/P_h ; its 70M sweep uses $\kappa = 0, .01, .05, .1$ in the matched latency sweep, with the additional $\kappa = 0.5$ endpoint reported separately in Appendix D.1. At 70M, T_4 and T_7 use P_h or P_{all} ; at 410M, only P_{all} . Parameter grids are given in Table 1. We evaluate the final checkpoint, without validation-based selection. The main figures measure loss and model-wide sparsity on the same validation sequences (Appendix B).

Figure 7 shows the training trajectories. They do not establish convergence, and the lower learning rates at 410M limit comparisons that attribute loss differences to model size alone.

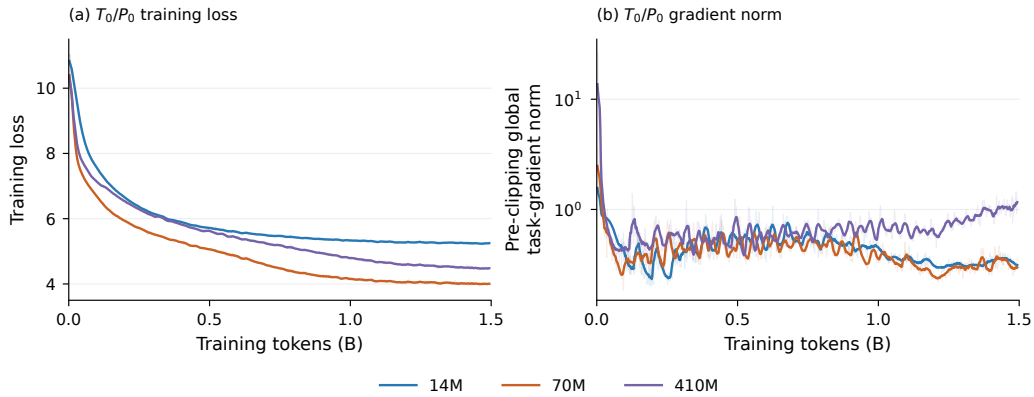


Figure 7: **Base-model (T_0/P_0) training across model sizes.** (a) Training loss and (b) gradient norm before clipping (logarithmic scale). Faint lines show individual updates; solid lines show centered nine-update means, with shorter windows at the ends. Gradient norms are computed after accumulation and are not normalized by parameter count.

D COMPLETE RESULTS AND SUPPORTING DIAGNOSTICS

D.1 DETAILED TRAINED RESULTS

Tables 8–11 report all 84 trained conditions, including settings outside the favorable quality–sparsity trade-off. Loss and model-wide sparsity use the same definitions and final checkpoints as the main figures, with one training seed and complete validation per condition. Values are rounded.

Table 8: **Additional 14M ablations.** Pressure at h (top) and thresholding without pressure (bottom). Controls and T_2/P_h appear in Table 10.

λ	T_1/P_1 (L1)		T_1/P_1 (OL1)	
	Loss	$\mathcal{S}_{\text{model}}$ (%)	Loss	$\mathcal{S}_{\text{model}}$ (%)
0.05	5.2061	3.142	5.1981	3.145
0.1	5.1655	3.336	5.1594	3.339
0.5	5.1127	3.788	5.1102	3.768
1	5.1023	3.949	5.1210	3.938
κ	T_4/P_0		T_7/P_0	
	Loss	$\mathcal{S}_{\text{model}}$ (%)	Loss	$\mathcal{S}_{\text{model}}$ (%)
0	5.4705	7.212	5.4684	7.218
0.01	5.4665	7.414	5.4588	7.618
0.05	5.4341	8.206	5.4379	9.127
0.1	5.4196	8.953	5.4287	10.425
0.5	5.6597	10.216	5.7029	15.387

The additional 70M T_2/P_h checkpoint at $\kappa = 0.5$ was measured in a later session with the same frozen optimized kernel. It reaches 15.427% sparsity at loss 4.8380, with latency reduced from 1.696 ms for the original port to 1.214 ms. Table 9 reports Base and $\kappa = 0.1$ repeated in that session: optimized latency changes by 2.19% and 0.20%, respectively, while native Base changes by 2.80%. Figure 5 retains the original 26 timings and marks this later endpoint separately, without pooling or rescaling.

Table 9: **Later 70M measurement and session references.** The frozen optimized kernel is evaluated on the additional T_2/P_h , $\kappa = 0.5$ checkpoint, with Base and $\kappa = 0.1$ repeated in the same later RTX 5090 session. Each row uses three fresh processes, 1,344 timings per implementation and full-validation numerical checks. Earlier rows are retained for comparison; sessions are not pooled.

Checkpoint	Session	Native (ms)	Port (ms)	Optimized (ms)
Base	Earlier	1.611	3.325	2.732
	Later	1.656	3.314	2.792
T_2/P_h , $\kappa = 0.1$	Earlier	1.662	2.258	1.956
	Later	1.721	2.259	1.959
T_2/P_h , $\kappa = 0.5$	Later	1.720	1.696	1.214

Table 10: **14M/70M results.** OL1 pressure uses $\lambda = b = 1$. The original 26 70M checkpoints share one RTX 5090 session (native Base: 1.611 ms). The T_2/P_h , $\kappa = 0.5$ row uses a later session with the same frozen kernel and protocol; its repeated references are given in Table 9. Absolute timings are not pooled or rescaled across sessions.

Recipe	κ	14M		70M		70M latency (ms)	
		Loss	$\mathcal{S}_{\text{model}}$ (%)	Loss	$\mathcal{S}_{\text{model}}$ (%)	Port	Optimized
T_0/P_0 (Base)	—	5.2086	0.000	4.0998	0.000	3.325	2.732
T_1/P_0 (ReLU)	—	5.2696	2.714	4.2227	10.066	3.292	2.717
T_2/P_h	0	5.1286	4.631	4.1287	14.630	2.730	2.504
	0.01	5.1306	4.717	4.1499	14.699	2.684	2.459
	0.05	5.1344	5.004	4.1925	15.030	2.409	2.137
	0.1	5.1511	5.175	4.1823	15.212	2.258	1.956
	0.5	5.5363	5.339	4.8380	15.427	1.696	1.214
T_4/P_h	0	5.2157	8.409	4.6065	25.792	2.597	2.383
	0.01	5.2045	8.586	4.6185	25.971	2.555	2.365
	0.05	5.1956	9.036	4.5964	26.566	2.229	1.892
	0.1	5.2287	9.343	4.7020	27.218	1.954	1.573
	0.5	5.7227	10.227	5.3057	29.236	1.593	1.088
T_4/P_{all}	0	5.4582	7.810	4.8053	25.673	2.774	2.599
	0.01	5.4583	8.587	4.8227	26.224	2.689	2.497
	0.05	5.4898	10.456	4.8739	28.490	2.286	2.023
	0.1	5.5483	11.394	4.9437	30.574	2.042	1.687
	0.5	6.0380	12.713	5.3895	35.596	1.598	1.105
T_7/P_h	0	5.2198	8.399	4.6508	25.786	2.624	2.412
	0.01	5.1981	8.856	4.6122	26.009	2.540	2.338
	0.05	5.1950	10.126	4.6670	26.895	2.217	1.875
	0.1	5.2374	10.904	4.7189	27.779	1.944	1.562
	0.5	5.7320	16.664	5.3372	32.233	1.590	1.088
T_7/P_{all}	0	5.4802	7.054	4.9412	23.562	3.143	2.741
	0.01	5.4758	7.723	4.9364	24.226	3.098	2.732
	0.05	5.4628	9.863	4.9636	26.524	2.960	2.717
	0.1	5.4295	11.797	4.9501	28.694	2.869	2.678
	0.5	5.8294	27.483	5.2160	40.602	1.718	1.276

D.2 POST-HOC CLIPPING AT 14M

Figure 8 extends the base/ReLU comparison in Figure ?? to the trained checkpoints with retained post-hoc sweeps. The detailed view resolves the trade-offs near the trained models; the full-range view shows the loss cost of aggressive clipping. These are changes to fixed models at evaluation, rather than additional training conditions. The clipping protocol is the same as in Section 4. No corresponding sweep was collected for T_2/P_h , T_4/P_h or T_7/P_h .

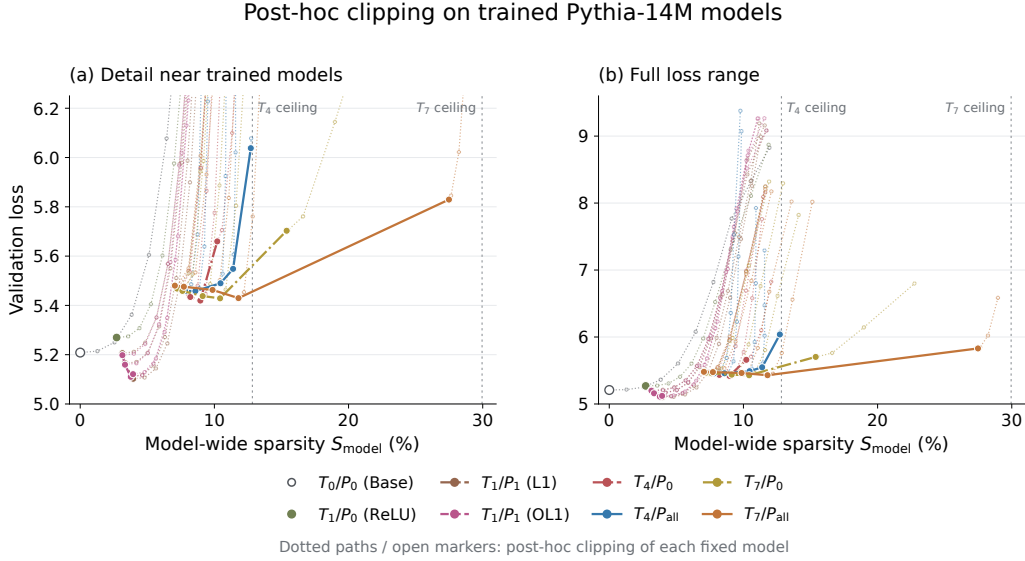


Figure 8: **Post-hoc clipping on trained Pythia-14M models.** Thirty trained checkpoints and ten clipping targets per checkpoint ($p = 0, 0.1, \dots, 0.9$). Both panels show the same settings, with different loss ranges. Prominent markers and recipe lines identify trained models; dotted paths with small open markers follow each fixed model through increasing clipping targets. Colors and recipe labels match the main figures, with the L1 comparison included explicitly. Each path starts at its measured $p = 0$ point. Lines connect evaluated settings, not fitted frontiers.

D.3 410M RESULTS

410M as a fixed-token stress test. Figure 9 retains all twelve trained 410M conditions and both complete control-clipping paths. At $\kappa = 0.5$, T_7/P_{all} reaches 80.62% model-wide sparsity, or 92.4% of the seven-site ceiling, at loss 5.121 versus the base model’s 4.547. This tests attainable sparsity under a larger architecture and the same token budget; it is not a quality-scaling experiment. The 410M seven-site ceiling is 87.25%, compared with 49.42% at 70M, so greater architectural reach also contributes to the larger observed model-wide sparsity.

The 410M base-model loss exceeds 70M’s 4.100. All sizes receive 712 optimizer steps and 1.493B input tokens, corresponding to 106.14, 21.20 and 3.68 tokens per parameter at 14M, 70M and 410M. The 410M schedule also has a lower peak learning rate (3×10^{-4} rather than 10^{-3}). These differences may contribute to the non-monotonic quality ordering, but their effects are not isolated. The training trajectories are retained in Figure 7. We therefore treat 410M as a limited stress test and use 14M/70M for detailed intervention and runtime analysis.

Table 11: **410M fixed-token stress test.** All twelve trained conditions; pressure uses OL1 with $\lambda = b = 1$. This size has no P_h or unpressured T_4/T_7 runs.

Recipe	κ	Loss	$\mathcal{S}_{\text{model}}$ (%)
T_0/P_0 (Base)	—	4.5475	0.011
T_1/P_0 (ReLU)	—	4.6513	21.050
T_4/P_{all}	0	5.6921	38.339
	0.01	5.6914	39.149
	0.05	5.6789	42.125
	0.1	5.6789	45.688
	0.5	5.1911	71.591
T_7/P_{all}	0	5.4263	41.121
	0.01	5.4331	41.478
	0.05	5.4679	44.053
	0.1	5.4908	48.998
	0.5	5.1209	80.616

Pythia-410M: fixed-token sparsity stress test

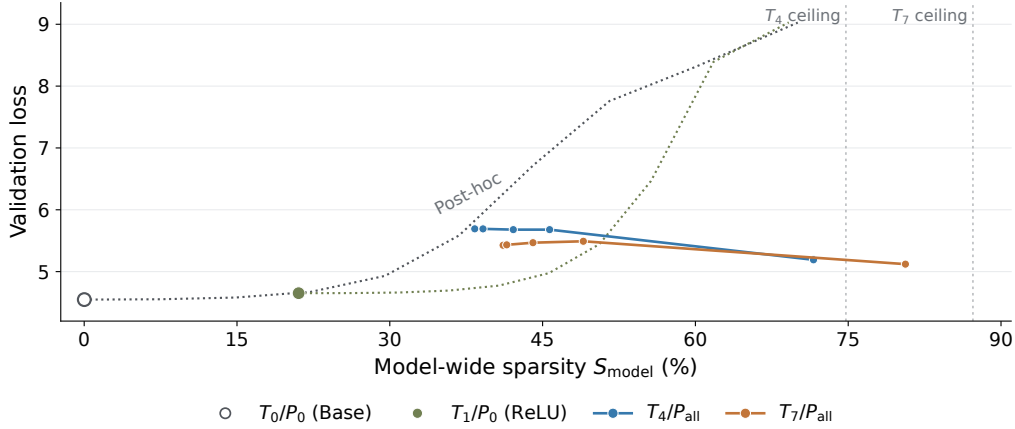


Figure 9: **410M quality-sparsity stress test.** All twelve trained conditions: Base, ReLU, and T_4/P_{all} and T_7/P_{all} at $\kappa = 0, .01, .05, .1, .5$. Dotted paths show all ten post-hoc targets for each fixed Base/ReLU control, including their full loss range. Vertical guides mark the T_4 and T_7 sparsity ceilings.

E KERNEL DEVELOPMENT AND EVALUATION

E.1 EXECUTION MECHANISMS

The specialized implementation preserves the trained weights and thresholding, while fusing operations and exploiting zero activations. At h, z , *sparse execution* includes both skipping empty activation tiles and replacing matrix instructions with scalar arithmetic for rows with very few nonzeros. The FFN-down and attention-output projections share a fused kernel; the vocabulary projection remains dense. Table 12 distinguishes work avoided before and after operand loads.

In Table 4, *bypass* is omitted MMA instructions divided by issued plus omitted instructions, pooling counts across layers and all 338 validation sequences. Counts include padded work and, at h, z , matrix instructions replaced by scalar arithmetic. Bypass therefore differs from $\mathcal{S}_{\text{model}}$, the model-wide fraction of zero-operand scalar products, and does not measure the fraction of latency saved.

Table 12: **Sparse execution rules in the 14M and initial 70M implementations.** MMA denotes a matrix multiply-accumulate instruction. Tile dimensions are tokens by input features. All zero checks execute during the forward pass.

Operation	Execution rule	Work avoided
QKV, FFN-up (a, m)	Check activation fragments after both operands are loaded.	Skip matrix instructions for empty fragments; retain operand reads.
FFN-down (h), attention-output (z)	Check 8×16 tiles before loading weights. Rows with at most two nonzeros use scalar arithmetic when numerical guards allow; other rows use the dense matrix path.	Empty tiles avoid weight reads and matrix instructions. Scalar rows read only weights paired with nonzero activations.
Attention (QK, PV)	Check operand fragments after loading them.	Skip matrix instructions if either fragment is zero; retain operand reads, masking and softmax.

E.2 TIMING AND NUMERICAL VALIDATION

Measurements use an RTX 5090, BF16, batch one, 2,048-token causal sequences, and full vocabulary logits for every token (PyTorch 2.11.0, Transformers 5.12.1, CUDA 12.8). This is full-sequence inference with fixed weights and shapes; cached decoding and concurrent serving are outside the evaluated scope. Final checkpoints were checked on all 338 validation sequences in three independent processes. Relative to the unmodified eager reference, acceptance required finite outputs, $|y - \hat{y}| \leq 0.25 + 0.02|\hat{y}|$ for every logit, relative ℓ_2 error at most 0.02 per sequence, and absolute mean validation-loss difference at most 0.001. Exact agreement in the 14M control below is stronger than these general tolerances.

PyTorch/SDPA and specialized implementations use CUDA graphs with identical inputs. We report geometric means of synchronized host timings over 64 fixed validation sequences, seven randomized paired passes and three independent processes. Timing includes embeddings, transformer blocks, final normalization, vocabulary logits, thresholding and all input-dependent sparsity checks. Compilation, static weight preparation, graph capture and equal input staging are excluded. Activations are recomputed on every forward pass.

Cross-recipe speedups use one PyTorch/SDPA Base reference per model size and measurement session: $t_{\text{PyTorch}, T_0} / P_0 / t_{\text{specialized}, r}$. The 14M reference is 0.65544 ms; Figure ??’s Base marker instead uses specialized execution. The 70M references are reported in Table 9. Timings from separate sessions are not pooled. The controls below instead compare execution configurations of the same checkpoint, with their denominators stated explicitly.

E.3 14M EXECUTION CONTROLS

Table 4 disables individual sparse paths at fixed weights, thresholds and fusion. Its ranges cover all off/on pairs of three process means per mode; effects are conditional and non-additive. The common 14M implementation was not tuned separately for each recipe: it retained attention skipping even though disabling it reduced latency in the historical grid. These tests measure full-model effects, not separate memory-read, inspection or arithmetic costs.

Table 13 tests the narrower T_2/P_h checkpoint at $\kappa = 0.1$. Disabling a site’s sparse execution removes both empty-tile skipping and scalar substitution, retaining the dense matrix path within the same fused kernel. Thresholding and all other paths remain active. Full-validation diagnostics confirm identical h, z values and zero masks in all six layers; logits agree exactly on all 338 sequences in three processes.

Joint savings ($D - A$) are $82.7 \mu\text{s}$, or 12.9% of D ’s latency (12.8–13.0% across processes). Disabling h alone adds $89.7 \mu\text{s}$. Enabling z saves $4.2 \mu\text{s}$ with h on but adds $7.0 \mu\text{s}$ with h off, showing why the individual effects cannot be added. The original 14M implementation, before adding independent execution switches, takes 0.5614 ms (12.5% below D). Its $2.6 \mu\text{s}$ offset from A matters when interpreting the smaller z effect.

Table 13: **14M T_2/P_h execution control at $\kappa = 0.1$.** All configurations retain thresholding. Full-model geometric-mean latencies follow the timing protocol above; sparse execution includes skipping and scalar substitution.

Configuration	h sparse execution	z sparse execution	Latency (ms)
A: full execution	On	On	0.5588
B: disable h	Off	On	0.6485
C: disable z	On	Off	0.5630
D: disable both	Off	Off	0.6415

PyTorch/SDPA on this same checkpoint takes 0.7018 ms; A reduces latency by 20.4%, with $D - A$ accounting for 57.9% of that gap. The remainder includes fusion and other implementation differences. This share does not apply to the historical comparison against the Base checkpoint.

E.4 70M ADAPTATION AND EXECUTION CONTROL

The initial 70M implementation retained the 14M execution rules; its best sparse recipe reaches only $1.013\times$ the native Base speed. Replacing its h, z kernel with PyTorch operations removed almost the entire latency deficit on the Base checkpoint, localizing the overhead to these projections. The dense matrix path pads eight rows to sixteen and repeats activation inspection across more output tiles at 70M. These are plausible sources of overhead; their individual costs were not isolated.

We selected the optimized implementation on sixteen fixed training sequences and fixed it before full-validation evaluation. It uses a row-sparsity prepass for h, z , expands scalar execution to more sparse rows and changes the matrix tiling. The a, m projections use PyTorch, attention uses a dense tiled kernel, and the vocabulary projection uses a tuned CUTLASS kernel. Thus dense operations also contribute to the improvement. Table 14 compares the implementations on three representative checkpoints; Table 10 gives the full grid.

Table 14: **70M implementation comparison.** Full-model geometric-mean latencies from the same RTX 5090 session. Each row holds the checkpoint and thresholding fixed across all three implementations; PyTorch/SDPA is the same-checkpoint reference.

Checkpoint	PyTorch/SDPA (ms)	Initial 70M (ms)	Optimized 70M (ms)
T_0/P_0 (Base)	1.611	3.325	2.732
$T_2/P_h, \kappa = 0.1$	1.662	2.258	1.956
$T_7/P_h, \kappa = 0.5$	1.824	1.590	1.088

At the lower-loss $T_2/P_h, \kappa = 0.1$, optimization reduces latency from 2.258 to 1.956 ms, still above the 1.662 ms PyTorch reference on the same checkpoint.

Table 15 separates two controls on optimized 70M execution. Disabling sparse h, z execution removes skipping and scalar substitution, retaining the dense matrix path within the same fused kernel. All modes pass the full-validation numerical criteria above; bitwise agreement was not established.

Table 15: **70M T_7/P_h execution controls at $\kappa = 0.5$.** Full-model geometric means with weights, thresholding and other optimized components fixed. The session differs from Table 14’s.

Execution configuration	Latency (ms)
Optimized implementation, sparse h, z execution enabled	1.128
Same implementation, sparse h, z execution disabled	2.704
Native PyTorch h, z replacement	1.365

Enabling sparse h, z execution reduces latency by 58.3% against the same kernel’s dense path. This benefit is conditional on that inefficient dense reference. Against the faster native replacement, the reduction is 17.4%, including fusion and layout differences.